

Digital Control of an Inverted Pendulum Using Midori Potentiometer Feedback

Team Members: Guido De Santis, Paulina Arizpe-Romo, and Eduardo Najera
MECH 488

Abstract

This report presents the implementation and tuning of a digital PD controller for an inverted pendulum using the conditioned output of a Midori potentiometer as the feedback signal. The control objective was to obtain a small-displacement response near the upright equilibrium with damping ratio $\zeta = 0.707$ and 2% settling time $T_s = 0.5$ s. The controller was implemented in C++ on the myRIO platform. The best response was obtained at $f_s = 1$ kHz using equivalent voltage-domain gains $k_p = 0.105$ and $k_d = 0.007$, giving $\zeta \approx 0.772$ and $T_s \approx 0.481$ s. Additional overdamped, underdamped, and lower-sampling-frequency trials were collected to show tunability and sampling-rate effects.

Contents

1 Introduction and Objective

The objective of this lab was to demonstrate digital control of the inverted pendulum using the conditioned output of the Midori potentiometer as the feedback signal. In the previous analog-control implementation, the pendulum angle and angular velocity were processed using analog signal-conditioning and control circuitry. In this experiment, the analog position signal was instead read by the myRIO data-acquisition system, processed in software, and converted back into an analog command voltage sent to the servo amplifier.

The desired closed-loop response was specified as a second-order response with damping ratio

$$\zeta_{\text{target}} = 0.707$$

and 2% settling time

$$T_{s,\text{target}} = 0.5 \text{ s.}$$

The controller was tested using small perturbations about the upright equilibrium position, since the linearized inverted-pendulum model is only valid for small angular displacements. The Midori signal-conditioning circuit was designed to produce approximately $\pm 10 \text{ V}$ for $\pm 10^\circ$ of pendulum motion, so the myRIO analog input channel was configured for a $\pm 10 \text{ V}$ input range.

The experiment had three main goals. First, we tuned a digital controller to meet the target settling-time and damping-ratio specifications. Second, we demonstrated the effect of varying the damping gain by collecting both overdamped and underdamped responses. Third, we compared controller behavior at two different sampling frequencies to observe the effect of sample period on the measured transient response. Because this project was completed for MECH 488, encoder-based control was not required.

2 Experimental Setup

The control loop used the following signal chain:

Midori potentiometer $\rightarrow$ analog position-conditioning circuit $\rightarrow$ myRIO analog input $\rightarrow$ digital controller $\rightarrow$ myRIO

Only the position signal from the Midori potentiometer was used as the measured feedback variable. The digital controller was implemented in C++ in the `control_midori` function. This function was called once per sample by the existing myRIO control framework. At each time step, the function received the current Midori voltage and returned the command voltage to be applied to the servo amplifier.

The feedback sign was set to

$$\text{feedback_sign} = 1.0$$

because the position-conditioning circuit used a non-inverting amplifier for the position amplification stage. Therefore, the polarity of the conditioned Midori signal matched the controller sign used in the final code. If the position amplifier had inverted the signal, the feedback sign would have needed to be reversed.

A second-order Butterworth low-pass filter was applied to the Midori voltage before the control law. This was done because derivative action in a PD controller is sensitive to high-frequency measurement noise. Filtering the measured position reduced noise-driven command oscillations while still preserving the lower-frequency pendulum motion needed for feedback control.

3 Controller Design and Supporting Calculations

3.1 Design target

For a standard underdamped second-order system, the approximate 2% settling-time relationship is

$$T_s \approx \frac{4}{\zeta\omega_n}.$$

Using the design specifications,

$$\zeta = 0.707, \quad T_s = 0.5 \text{ s},$$

the target natural frequency is

$$\omega_n = \frac{4}{\zeta T_s} = \frac{4}{(0.707)(0.5)} \approx 11.3 \text{ rad/s}.$$

This natural frequency and damping ratio defined the desired transient response for small displacements about the upright equilibrium.

3.2 Sensor scaling

The analog position-conditioning circuit was designed to produce

$$\pm 10 \text{ V}$$

for

$$\pm 10^\circ$$

of pendulum angular displacement. Since

$$10^\circ = \frac{10\pi}{180} \approx 0.1745 \text{ rad},$$

the position sensor scale is

$$S_\theta = \frac{10 \text{ V}}{0.1745 \text{ rad}} \approx 57.3 \frac{\text{V}}{\text{rad}}.$$

Thus, the pendulum angle can be computed from the Midori voltage as

$$\theta = \frac{V_{\text{Midori}} - V_{\text{zero}}}{57.3}.$$

Here V_{zero} is the conditioned Midori voltage when the pendulum is upright. In the final implementation, the code was written in an equivalent voltage-domain form, where the conversion from Midori voltage to radians and from motor current to amplifier voltage were absorbed into the tuned gains. This made experimental tuning more direct while preserving the same PD-control structure.

3.3 Physical control law

The physical continuous-time control law for the inverted pendulum is

$$i_{\text{cmd}} = -K_\theta \theta - K_\omega \dot{\theta},$$

where K_θ is the proportional position gain in A/rad and K_ω is the derivative, or velocity, gain in A/(rad/s).

The servo amplifier gain is

$$K_s = 0.6 \frac{\text{A}}{\text{V}}.$$

Therefore, the myRIO command voltage corresponding to a desired motor current is

$$V_{\text{cmd}} = \frac{i_{\text{cmd}}}{K_s}.$$

Combining the sensor conversion and servo-amplifier conversion gives an equivalent voltage-domain proportional gain

$$k_p = \frac{K_\theta}{K_s S_\theta}.$$

Similarly, the equivalent voltage-domain derivative gain is

$$k_d = \frac{K_\omega}{K_s S_\theta}.$$

From the analog control lab, the measured/estimated position gain was approximately

$$K_\theta \approx 7.15 \frac{\text{A}}{\text{rad}}.$$

Using the sensor and servo-amplifier conversions, the corresponding initial voltage-domain proportional gain is

$$k_{p,\text{initial}} = \frac{7.15}{(0.6)(57.3)} \approx 0.208.$$

The initial theoretical derivative gain used for comparison was approximately

$$K_\omega \approx 0.60 \frac{\text{A}}{\text{rad/s}},$$

which corresponds to

$$k_{d,\text{initial}} = \frac{0.60}{(0.6)(57.3)} \approx 0.0175 \text{ s}.$$

The final experimentally tuned voltage-domain gains were lower:

$$k_p = 0.105, \quad k_d = 0.007.$$

The corresponding effective physical gains are

$$K_{\theta,\text{eff}} = k_p K_s S_\theta = (0.105)(0.6)(57.3) \approx 3.61 \frac{\text{A}}{\text{rad}},$$

and

$$K_{\omega,\text{eff}} = k_d K_s S_\theta = (0.007)(0.6)(57.3) \approx 0.241 \frac{\text{A}}{\text{rad/s}}.$$

These final values were lower than the initial theoretical analog gains because the real system included effects not fully captured by the simplified model, including sensor filtering, motor/amplifier dynamics, friction, saturation, sign convention, and uncertainty in the exact signal-conditioning scale.

3.4 Digital PD emulation

The continuous-time PD controller has the form

$$H(s) = k_p + k_d s.$$

To implement this controller digitally, the derivative operator was approximated using the bilinear, or Tustin, transformation:

$$s \rightarrow \frac{2}{T} \frac{z - 1}{z + 1},$$

where

$$T = \frac{1}{f_s}$$

is the sampling period.

Substituting the Tustin expression into the PD controller gives

$$H(z) = k_p + k_d \left(\frac{2}{T} \frac{z - 1}{z + 1} \right).$$

Combining terms,

$$H(z) = \frac{\left(k_p + \frac{2k_d}{T} \right) z + \left(k_p - \frac{2k_d}{T} \right)}{z + 1}.$$

Dividing numerator and denominator by z ,

$$H(z) = \frac{A + Bz^{-1}}{1 + z^{-1}},$$

where

$$A = k_p + \frac{2k_d}{T},$$

and

$$B = k_p - \frac{2k_d}{T}.$$

Since

$$H(z) = \frac{U(z)}{E(z)},$$

we have

$$U(z)(1 + z^{-1}) = E(z)(A + Bz^{-1}).$$

Converting back to the time domain gives

$$u_k + u_{k-1} = Ae_k + Be_{k-1}.$$

Therefore, the implemented command voltage is

$$u_k = Ae_k + Be_{k-1} - u_{k-1}.$$

This is the equation implemented in the C++ controller. The variables e_k , e_{k-1} , and u_{k-1} correspond to the current error, previous error, and previous command voltage, respectively.

For the nominal final controller at $f_s = 1000$ Hz,

$$T = 0.001 \text{ s}, \quad k_p = 0.105, \quad k_d = 0.007.$$

Thus,

$$A = 0.105 + \frac{2(0.007)}{0.001} = 14.105,$$

and

$$B = 0.105 - \frac{2(0.007)}{0.001} = -13.895.$$

For the lower sampling-rate trial at $f_s = 500$ Hz,

$$T = 0.002 \text{ s.}$$

Using the same tuned gains,

$$A = 0.105 + \frac{2(0.007)}{0.002} = 7.105,$$

and

$$B = 0.105 - \frac{2(0.007)}{0.002} = -6.895.$$

These coefficients must be recomputed whenever the sampling rate is changed.

4 Implementation Method

The C++ implementation was placed in the `control_midori` function in `src/myrio/pendulum.cpp`. The myRIO framework called this function repeatedly at the selected sampling rate. The function received the current controller time and the measured Midori voltage, then returned the command voltage to be sent to the servo amplifier.

The sample rate was changed in the `main` function using the line

```
1 Frequency sample_rate = hertz(1000);
```

for the nominal 1 kHz case, and

```
1 Frequency sample_rate = hertz(500);
```

for the lower-sampling-frequency case. The code was rebuilt and uploaded to the myRIO after each change.

The final controller used the following code structure:

1. Read the conditioned Midori voltage.
2. Filter the measured Midori voltage using a second-order Butterworth filter.
3. Compute the feedback error using the appropriate sign for the non-inverting position amplifier.
4. Compute the Tustin PD coefficients A and B from the current sample period.
5. Apply the difference equation $u_k = Ae_k + Be_{k-1} - u_{k-1}$.
6. Clamp the command voltage to the safe ± 10 V range.
7. Store the current error and command voltage for use at the next sample.

5 Response Analysis Method

The response was analyzed using the measured filtered Midori error signal. For each release, the response was shifted so that time zero corresponded approximately to the first peak or release point. The final value was estimated from the steady-state portion of the response. The initial amplitude was computed as

$$A_{\text{resp}} = |y_{\text{peak}} - y_{\text{final}}|.$$

The 2% settling band was then defined as

$$\pm 0.02 A_{\text{resp}}$$

about the final value. The measured settling time was the first time after release at which the response entered and remained inside this 2% band.

For underdamped responses, the logarithmic decrement method can be used to estimate damping ratio. If X_1 and X_2 are successive same-side peak amplitudes measured relative to the final value, then

$$\delta = \ln \left(\frac{X_1}{X_2} \right).$$

The corresponding damping ratio is

$$\zeta = \frac{\delta}{\sqrt{4\pi^2 + \delta^2}}.$$

For the nominal and 500 Hz cases, the damping ratio was obtained by fitting the measured decay to a second-order response. This gave $\zeta \approx 0.772$ for the nominal 1 kHz case and $\zeta \approx 0.779$ for the 500 Hz case.

6 Results

6.1 Best overall response

The best overall tuning was found to be

```
1 const double kp = 0.105;  
2 const double kd = 0.007;
```

using the response recorded in `trial17.csv`. From the selected response segment, the experimentally extracted values were

$$\zeta_{\text{exp}} \approx 0.772, \quad T_{s,\text{exp}} \approx 0.481 \text{ s}.$$

The corresponding percent errors relative to the design specifications are

$$\% \text{ error in } \zeta = \frac{0.772 - 0.707}{0.707} \times 100 \approx 9.2\%,$$

and

$$\% \text{ error in } T_s = \frac{0.481 - 0.500}{0.500} \times 100 \approx -3.8\%.$$

Thus, the final response was slightly more damped than the target, but it was very close to the desired settling-time requirement.

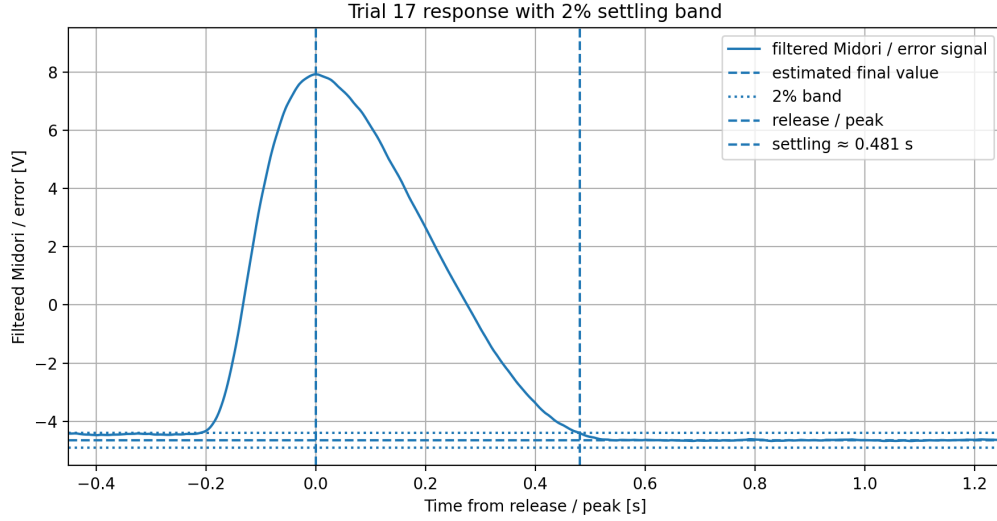


Figure 1: Measured response for the final chosen controller, $k_p = 0.105$ and $k_d = 0.007$. The dashed horizontal lines show the 2% settling band and the vertical dashed line marks the measured settling time, approximately 0.481 s.

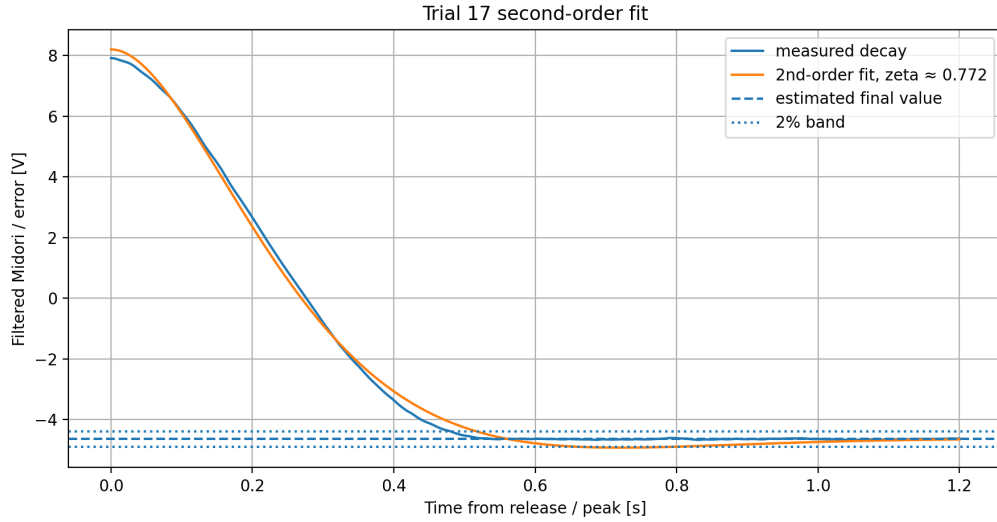


Figure 2: Second-order fit for the final chosen controller. The fitted damping ratio is approximately $\zeta \approx 0.772$, compared with the target value 0.707.

6.2 Overdamped example

A clearly overdamped response was obtained with

```
1 const double kp = 0.1;
2 const double kd = 0.02;
```

using the response in `trial23.csv`. For the selected release,

$$t_0 = 26.139 \text{ s}, \quad y_{\text{peak}} = 6.433 \text{ V}, \quad y_{\text{final}} \approx -2.069 \text{ V}.$$

Therefore, the response amplitude was

$$A_{\text{resp}} \approx 6.433 - (-2.069) = 8.502 \text{ V},$$

and the 2% settling band was

$$0.02A_{\text{resp}} \approx 0.170 \text{ V}.$$

The measured 2% settling time was approximately

$$T_s \approx 2.44 \text{ s}.$$

The overshoot past the final value was very small, about 1.93%, and the response returned slowly without visible oscillation. This behavior is characteristic of an overdamped response, so it is appropriate to classify this case as

$$\zeta > 1.$$

Because the response is monotonic and heavily overdamped, a precise numerical value of ζ is not as reliable or important as the qualitative classification.

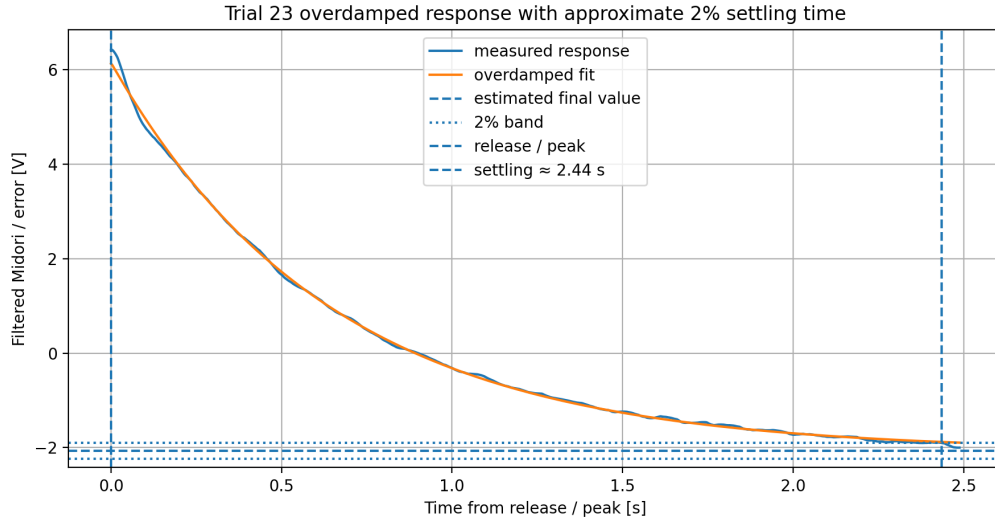


Figure 3: Overdamped response for $k_p = 0.1$ and $k_d = 0.02$. The response is slow and nearly monotonic, with very little overshoot and a much longer settling time, approximately 2.44 s, than the nominal design.

6.3 Underdamped example

A clearly underdamped response was obtained with

```
1 const double kp = 0.09;
2 const double kd = 0.001;
```

using the response in `trial22.csv`. This response did not behave like a clean small-signal second-order transient. Instead, it exhibited large oscillations and saturation near the measurement limits. The major observed excursions were approximately

$$+7.1 \text{ V}, \quad -10.1 \text{ V}, \quad +10.9 \text{ V}.$$

The response crossed the equilibrium and overshoot strongly in both directions, indicating that the damping was far too low. Because it also approached the measurement and actuator limits, this trial did not provide a reliable small-signal estimate of settling time or damping ratio. Nevertheless, it is an excellent qualitative example of underdamped behavior. Therefore, this case is classified as

$$\zeta < 1,$$

with large overshoot and oscillation. In practice, the response was so lightly damped that it appeared unstable-looking and violated the small-angle assumption used for the nominal design.

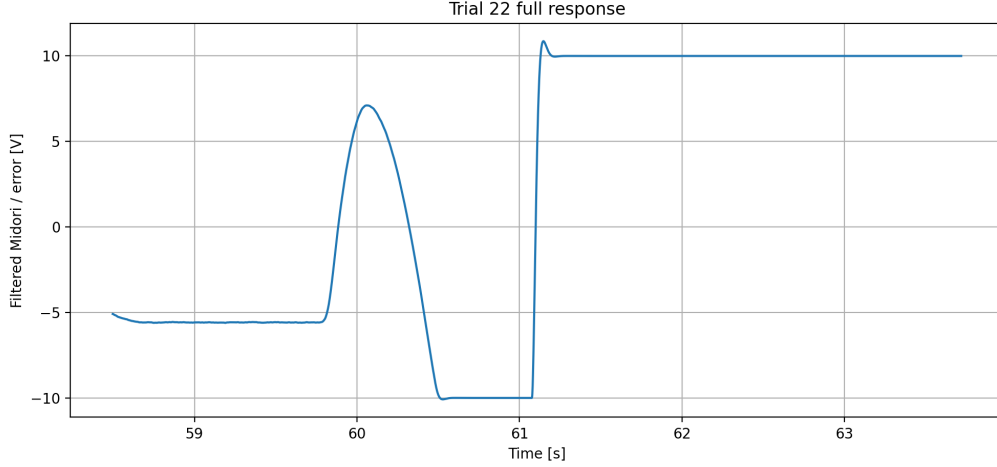


Figure 4: Underdamped response for $k_p = 0.09$ and $k_d = 0.001$. The response overshoot strongly and swung from about +7.1 V to -10.1 V and then to +10.9 V, indicating a clearly underdamped response.

6.4 Lower sampling-frequency trial

A separate trial was collected at a lower sampling frequency of

$$f_s = 500 \text{ Hz}, \quad T_{\text{sample}} = 2 \text{ ms}.$$

The same nominal tuning was used:

$$k_p = 0.105, \quad k_d = 0.007.$$

This trial was analyzed using the same response-fitting and 2% settling-time method as the 1 kHz trials. For the selected release,

$$t_0 = 15.026 \text{ s}, \quad y_{\text{peak}} = 9.562 \text{ V}, \quad y_{\text{final}} \approx -4.617 \text{ V}.$$

The response amplitude was

$$A_{\text{resp}} = 9.562 - (-4.617) = 14.179 \text{ V},$$

and the 2% settling band was

$$0.02A_{\text{resp}} \approx 0.284 \text{ V}.$$

The measured 2% settling time was

$$T_s \approx 0.506 \text{ s},$$

and the fitted damping ratio was

$$\zeta \approx 0.779.$$

Compared with the design target of $\zeta = 0.707$ and $T_s = 0.5 \text{ s}$, the 500 Hz trial was very close in settling time and slightly more damped than desired.

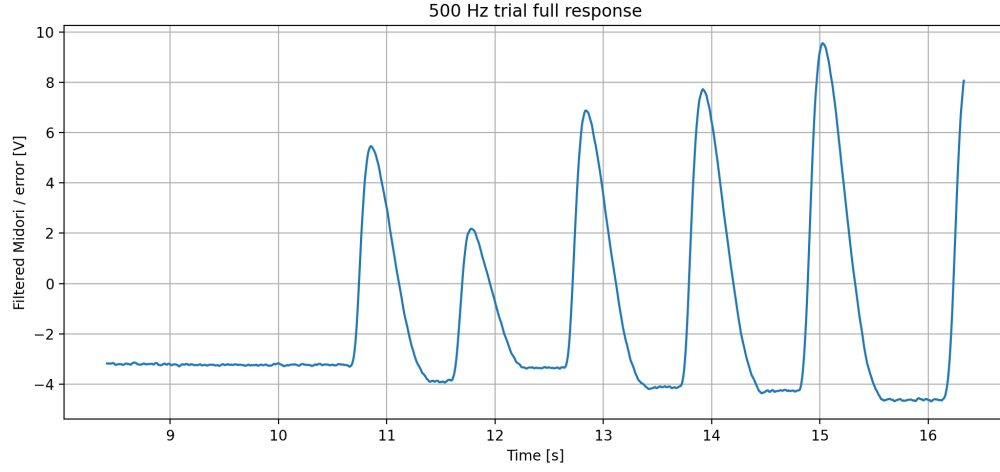


Figure 5: Full 500 Hz response. The sample period is 2 ms, which is still much smaller than the pendulum settling-time scale.

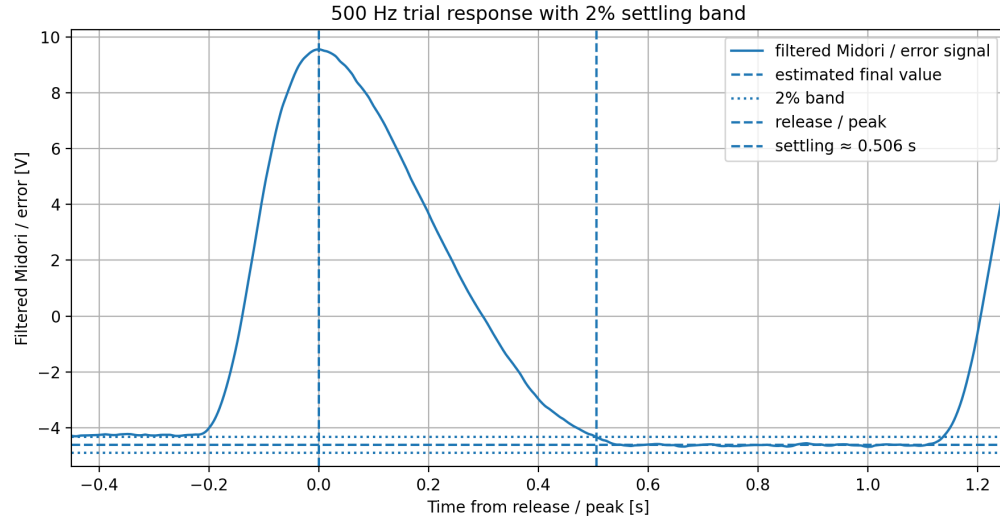


Figure 6: 500 Hz response with the 2% settling band. The measured settling time for the selected release was approximately 0.506 s.

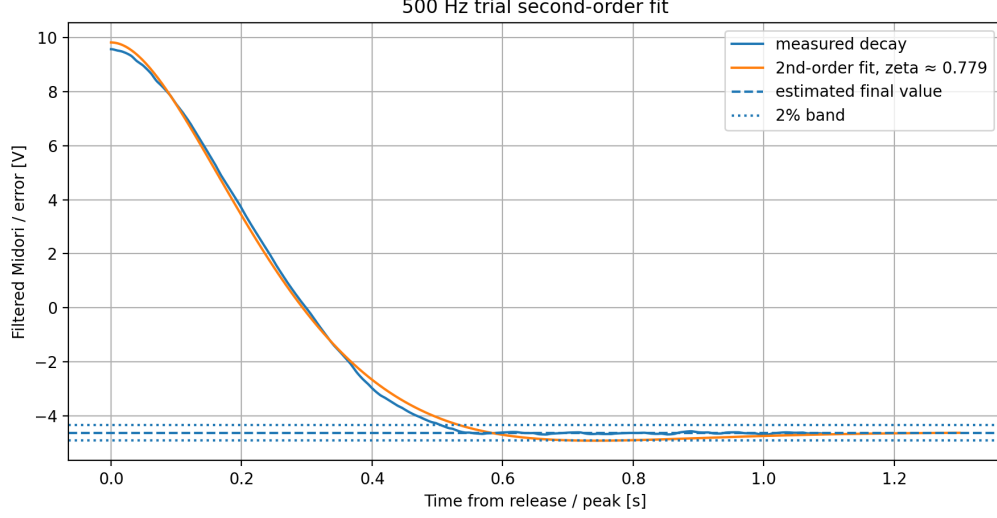


Figure 7: Second-order fit to the 500 Hz response. The fitted damping ratio was approximately $\zeta \approx 0.779$.

7 Discussion

The final 1 kHz controller setting,

$$k_p = 0.105, \quad k_d = 0.007,$$

gave the best overall match to the required response. The measured damping ratio was

$$\zeta_{\text{exp}} \approx 0.772,$$

and the measured 2% settling time was

$$T_{s,\text{exp}} \approx 0.481 \text{ s}.$$

This response was slightly more damped than the target value of $\zeta = 0.707$, but it met the settling-time requirement closely.

Changing k_d had the expected qualitative effect on the response. Increasing k_d produced a more overdamped response, reducing overshoot but increasing settling time. The overdamped case with

$$k_p = 0.1, \quad k_d = 0.02$$

settled slowly, with an approximate settling time of 2.44 s. Decreasing k_d produced a more underdamped response. The underdamped case with

$$k_p = 0.09, \quad k_d = 0.001$$

showed large oscillation and saturation, indicating that the damping was too low.

The 500 Hz sampling case produced a response close to the 1 kHz nominal case. This indicates that the pendulum dynamics were slow enough that both sampling periods were adequate. However, the sampling period still enters directly into the digital control coefficients,

$$A = k_p + \frac{2k_d}{T}, \quad B = k_p - \frac{2k_d}{T},$$

so the coefficients must be recomputed whenever the sampling rate is changed.

Several practical issues affected the tuning process. First, the feedback sign depended on the polarity of the analog position-conditioning circuit. Because our position amplifier was non-inverting, a feedback sign of +1.0 was used. Second, filtering was necessary because derivative control is sensitive to measurement noise. Third, large displacements violated the small-angle assumption and could cause saturation near the ± 10 V measurement or command limits. For this reason, the final performance analysis used small displacements near the upright position.

8 Summary

Table 1: Summary of the controller settings and sampling-frequency cases used in the report.

Case	f_s	k_p	k_d	Damping characterization	Settling time
Best overall / nominal	1 kHz	0.105	0.007	$\zeta \approx 0.772$	≈ 0.481 s
500 Hz sampling trial	500 Hz	0.105	0.007	$\zeta \approx 0.779$	≈ 0.506 s
Overdamped example	1 kHz	0.100	0.020	$\zeta > 1$	≈ 2.44 s
Underdamped example	1 kHz	0.090	0.001	$\zeta < 1$	not reliably extracted

9 Video Links

The following videos document the required pendulum demonstrations:

- Different sampling frequencies: <https://youtu.be/eVfbxd7lRl8>
- Underdamped response: <https://youtu.be/9ALoWhzv0xE>
- Overdamped response: <https://youtu.be/2RsIy3gENmQ>

10 Conclusion

At a sampling rate of 1 kHz, the final controller gains

$$k_p = 0.105, \quad k_d = 0.007$$

produced the best overall match to the required response. The measured damping ratio was approximately 0.772, and the measured 2% settling time was approximately 0.481 s. This was slightly more damped than the target value $\zeta = 0.707$, but the settling time was very close to the specified 0.5 s.

The additional trials successfully demonstrated the qualitative differences between overdamped and underdamped behavior. The overdamped example, $k_p = 0.1$ and $k_d = 0.02$, showed a slow, nearly monotonic return with a settling time of about 2.44 s. The underdamped example, $k_p = 0.09$ and $k_d = 0.001$, showed large oscillation and saturation, making it a clear example of insufficient damping. The 500 Hz sampling trial also remained close to the target response, giving $\zeta \approx 0.779$ and $T_s \approx 0.506$ s, showing that the lower sampling rate was still adequate for this mechanical response.

A Code Screenshots

The following screenshots document the implemented `control_midori` function used for the final Midori potentiometer feedback controller.

```
12  class MyPendulum : public IPendulum {
24      /// Part 1: Implement control with Midori potentiometer position feedback
25  double control_midori(double t, double midori_volts) override {
26      // Digital PD control using Midori potentiometer feedback.
27      // This version works directly in voltage-domain:
28      // Midori volts -> filtered error -> command voltage.
29
30      const double T = 1.0 / sample_freq;
31
32      // Voltage-domain gains.
33      // These already absorb:
34      // Midori volts -> radians
35      // controller current -> amplifier command voltage
36      const double kp = 0.105;
37      const double kd = 0.007;
38
39      // Flip this sign if the pendulum pushes itself away from vertical.
40      const double feedback_sign = 1.0;
41
42      const double A = kp + 2.0 * kd / T;
43      const double B = kp - 2.0 * kd / T;
44
45      static double last_error = 0.0;
46      static double last_control = 0.0;
47
48      // Use 40 Hz filter at 1000 Hz sampling, lower cutoff at 100 Hz sampling.
49      const double cutoff_hz = sample_freq >= 500.0 ? 20.0 : 10.0;
50      static Butterworth midori_filter(2, hertz(10), hertz(sample_freq));
51
52      double filtered_midori_volts = midori_filter.update(midori_volts);
53
54      double error = feedback_sign * filtered_midori_volts;
55
56      double command_voltage = A * error + B * last_error - last_control;
57
58      command_voltage = clamp(command_voltage, -10.0, 10.0);
59
60      last_error = error;
61      last_control = command_voltage;
```

Figure 8: First portion of the implemented `control_midori` function, including the sample-period definition, tuned gains, feedback sign, and filter setup.

```

41
42     const double A = kp + 2.0 * kd / T;
43     const double B = kp - 2.0 * kd / T;
44
45     static double last_error = 0.0;
46     static double last_control = 0.0;
47
48     // Use 40 Hz filter at 1000 Hz sampling, lower cutoff at 100 Hz sampling.
49     const double cutoff_hz = sample_freq >= 500.0 ? 20.0 : 10.0;
50     static Butterworth midori_filter(2, hertz(10), hertz(sample_freq));
51
52     double filtered_midori_volts = midori_filter.update(midori_volts);
53
54     double error = feedback_sign * filtered_midori_volts;
55
56     double command_voltage = A * error + B * last_error - last_control;
57
58     command_voltage = clamp(command_voltage, -10.0, 10.0);
59
60     last_error = error;
61     last_control = command_voltage;
62
63     plot("midori_volts", midori_volts);|
64     plot("filtered_midori_volts", filtered_midori_volts);
65     plot("error", error);
66     plot("command_voltage60", command_voltage);
67     plot("kp", kp);
68     plot("kd", kd);
69
70     return command_voltage;
71 }
72

```

Figure 9: Second portion of the implemented `control_midori` function, including the difference equation, voltage clamp, plotting calls, and returned command voltage.

B Commented Source Code

The following code was used for the Midori potentiometer feedback controller. The code is included here for documentation and reproducibility.

```

1  /// Part 1: Implement control with Midori potentiometer position feedback.
2  double control_midori(double t, double midori_volts) override {
3      // Digital PD control using Midori potentiometer feedback.
4      //
5      // The myRIO framework calls this function once per sample.
6      // Input:
7      //     t                = current controller time [s]
8      //     midori_volts     = conditioned Midori potentiometer voltage [V]
9      //
10     // Output:

```

```

11 //      command_voltage = voltage command sent to the servo amplifier
12 //      [V]
13 // The analog position circuit was designed for approximately
14 // +/-10 V at +/-10 degrees. In the final implementation, the
15 // voltage-to-radian conversion and servo-amplifier gain conversion
16 // are absorbed into the equivalent voltage-domain gains kp and kd.
17
18 const double T = 1.0 / sample_freq; // sample period [s]
19
20 // Equivalent voltage-domain controller gains.
21 // These correspond to the physical PD law after including:
22 //      Midori voltage -> pendulum angle
23 //      motor current  -> servo amplifier command voltage
24 //
25 // Best final tuning:
26 //      kp = 0.105
27 //      kd = 0.007
28 const double kp = 0.105;
29 const double kd = 0.007;
30
31 // Feedback sign.
32 // This is +1.0 because our position amplification stage was non-
33 //      inverting.
34 // If the signal-conditioning circuit inverted the position signal,
35 //      this
36 // sign would need to be changed.
37 const double feedback_sign = 1.0;
38
39 // Tustin / bilinear digital PD coefficients:
40 //      A = kp + 2*kd/T
41 //      B = kp - 2*kd/T
42 const double A = kp + 2.0 * kd / T;
43 const double B = kp - 2.0 * kd / T;
44
45 // Static variables retain their values between function calls.
46 // last_error stores e_{k-1}.
47 // last_control stores u_{k-1}.
48 static double last_error = 0.0;
49 static double last_control = 0.0;
50
51 // Low-pass filter for the Midori voltage.
52 // The derivative action is sensitive to sensor noise, so filtering
53 // helps reduce high-frequency command oscillations.
54 static Butterworth midori_filter(2, hertz(10), hertz(sample_freq));
55
56 // Filter the raw Midori voltage before using it in the controller.
57 double filtered_midori_volts = midori_filter.update(midori_volts);
58
59 // Compute the feedback error.
60 // Because the amplifier was non-inverting, feedback_sign = +1.0.
61 double error = feedback_sign * filtered_midori_volts;
62
63 // Digital PD difference equation:

```



```

62 //      u_k = A*e_k + B*e_{k-1} - u_{k-1}
63 double command_voltage = A * error + B * last_error - last_control;
64
65 // Limit the command to the safe output range of the myRIO/servo
66 // amplifier.
67 command_voltage = clamp(command_voltage, -10.0, 10.0);
68
69 // Store current values for the next sample.
70 last_error = error;
71 last_control = command_voltage;
72
73 // Plot variables in the GUI for debugging and data export.
74 plot("midori_volts", midori_volts);
75 plot("filtered_midori_volts", filtered_midori_volts);
76 plot("error", error);
77 plot("command_voltage", command_voltage);
78 plot("kp", kp);
79 plot("kd", kd);
80
81 return command_voltage;
82 }

```